Rapport projet de Big Data : analyse du marché de l'emploi des métiers de la data

Analyse des compétences et prédiction des salaires avec un pipeline Spark.

Dépôt GitHub : github.com/MarieJoelleWayou/BigData2026

1. Objectif et démarche

Le projet porte sur le Marché de l'Emploi dans le domaine de la data. La question est double : quelles compétences sont demandées aux profils data (Data Analyst, Data Engineer, Data Scientist, ingénieur ML…) et peut-on prédire le salaire proposé par une offre à partir de ses caractéristiques ?

Pour y répondre, nous avons construit une chaîne de traitement complète : collecte de 2 050 offres sur deux sources, stockage brut, nettoyage et analyse textuelle distribués avec PySpark, puis comparaison de trois modèles de régression PySpark ML. Le code est réparti en deux notebooks : webscrapping.ipynb (collecte) et traitement_spark_ml.ipynb (traitement et modélisation).

2. Architecture et flux de données

INGESTION (webscrapping.ipynb) :

API France Travail

OAuth2 + requests

1 045 offres

Welcome to the Jungle

Selenium + BeautifulSoup

1 246 offres

▼

Stockage brut :

JSON brut de l'API · CSV aplatis (schéma commun de 20 colonnes)

▼

Spark processing : traitement_spark_ml.ipynb

union des sources → correction des doublons → correction valeurs manquantes → conversion du salaire valeur annuel → création et sélection des variables → MapReduce top-k

▼

Zone nettoyée :

Parquet partitionné par source : 2104 offres

▼

Modeling : PySpark ML

Régression linéaire · Random Forest · Gradient Boosted Trees

Justification des formats

- **JSON brut** pour la réponse de l'API France Travail : c'est le format natif. Le conserver sans transformation permet de refaire tout le traitement si une règle de nettoyage change, sans rappeler l'API.
- **CSV** pour la version aplatie de chaque source : simple à produire depuis pandas, lisible par un humain pour vérifier la collecte, et lu directement par Spark.
- **Parquet** pour les données nettoyées : format colonnaire compressé, qui stocke le schéma avec les données (plus de ré-inférence des types) et permet à Spark de ne lire que les colonnes utiles à une requête.

Partitionnement

Les données nettoyées sont partitionnées par source. Les deux sources ont des formats et qualités différents ; ce découpage permet de relire ou de modifier une source sans toucher à l'autre, et Spark peut ignorer une partition entière lors d'un filtre sur la source. Avec environ 2 000 offres,

un partitionnement plus fin (par date ou par département) produirait surtout une multitude de petits fichiers, ce qui dégrade les performances de Spark.

3. Méthodologie

3.1 Acquisition des données

France Travail. L'accès passe par l'API officielle « Offres d'emploi v2 », avec un jeton OAuth2 obtenu par client credentials. Les identifiants sont stockés dans un fichier .env exclu du dépôt Git. Six mots-clés sont interrogés (data analyst, data engineer, data scientist, machine learning, analyste données, analyste quantitatif), par pages de 150 offres, avec une pause d'une seconde entre les appels. Chaque offre est aplatie (intitulé, description, contrat, expérience, salaire, lieu, compétences, formation…).

Welcome to the Jungle. Le site étant rendu en JavaScript, la collecte utilise Selenium puis BeautifulSoup. Pour chaque offre de la liste de résultats, le scraper ouvre la page de détail et lit les données structurées JSON-LD (salaire, formation, localisation) ainsi que la description. Le fichier obtenu en avril 2026 est conservé : depuis, le site a modifié sa structure HTML et le même code ne trouve plus aucune offre.

Les deux sources fournissent bien des données structurées (salaire, intitulé, contrat, lieu) et textuelles (descriptions), comme demandé.

3.2 Nettoyage distribué avec PySpark

Étape	Offres
Données brutes (union des deux sources)	2 050
Suppression des doublons d'identifiant	2 050
Suppression des doublons inter-sources (même description normalisée)	1 987
Suppression des offres sans description	1 975
Filtrage des offres hors périmètre data	1 858

- **Doublons inter-sources** : une même offre publiée sur les deux sites a deux identifiants différents ; elle est repérée par sa description normalisée (minuscules, sans ponctuation) avec une fenêtre row_number(), en gardant la version France Travail.
- **Intitulés WTTJ** : le titre du poste étant découpé en plusieurs balises HTML, le scraper n'en a souvent récupéré qu'un fragment (« Data », « Business », voire le badge « Recrute activement ! »). L'intitulé est reconstruit à partir de l'URL de l'offre, qui contient le titre complet (data-analyst-h-f_paris… → « Data Analyst »). Pour la même raison, la colonne entreprise de WTTJ n'est pas utilisée.
- **Salaire** : les libellés sont hétérogènes (« Annuel de 40000 Euros à 47000 Euros », « Mensuel de 1407 Euros », « Horaire de 12,31 Euros », « EUR 50000 - 55000 / an », « < 1,3K € par mois »). Le centre de la fourchette est ramené à un salaire annuel brut (×12 pour un mensuel, ×1 607 h pour un horaire). Les erreurs de saisie sont corrigées (un montant inférieur à 5 000 € « / an » est mensuel, un « mensuel » de 26 000 € est annuel) et les valeurs hors de [6 000 ; 200 000] € sont écartées.
- **Valeurs manquantes** : les variables catégorielles reçoivent la modalité « Non renseigné », car l'absence d'information est elle-même informative (une offre sans type de contrat est souvent moins détaillée). Les années d'expérience manquantes (49 %) sont imputées par la médiane dans le pipeline ML, avec un Imputer ajusté sur l'apprentissage seulement.
- **Périmètre** : les recherches par mots-clés de France Travail renvoient aussi des offres sans lien avec la data (électricien, agent de propreté…). On ne garde que les offres dont l'intitulé relève de la data, ou dont la description cite au moins trois compétences techniques.

3.3 Construction des variables

- **Catégorie de poste** déduite de l'intitulé (Data Analyst, Data Engineer, Data Scientist, ML / IA, BI, Management, Stage / Alternance, Autre) et **séniorité** (Junior, Senior / Confirmé, Lead / Manager).
- **Contrat**, **niveau de formation** (harmonisé entre les deux sources) et **région** (Île-de-France ou autres régions, d'après le code postal).
- **Années d'expérience**, lues dans le champ France Travail (« 3 An(s) », « Débutant accepté ») ou dans la description.

- **22 indicateurs de compétences** (Python, SQL, Spark, Power BI, dbt, Azure…), détectés par expressions régulières dans la description et les compétences listées.

3.4 Analyse NLP : procédure MapReduce

Les k mots les plus fréquents des descriptions sont extraits avec l'API RDD de Spark, selon le schéma MapReduce :

- **Map** (flatMap) : chaque description est découpée en mots ; chaque mot utile est émis sous la forme d'une paire (mot, 1). Les mots vides français et anglais (listes de StopWordsRemover) et le vocabulaire générique des annonces sont filtrés.
- **Shuffle** : Spark regroupe les paires de même clé sur une même partition.
- **Reduce** (reduceByKey) : les compteurs de chaque mot sont additionnés, avec un pré-agrégat local sur chaque partition.
- **Top-k** : takeOrdered(k) renvoie les k plus fréquents sans trier tout le RDD.

La même procédure, avec une clé composite (source, mot), compare le vocabulaire des deux sources.

3.5 Modélisation

La cible est le salaire annuel brut, connu pour 364 offres (20 %). Les données sont séparées en apprentissage (80 %) et test (20 %). Un pipeline PySpark ML commun (Imputer, StringIndexer, OneHotEncoder, VectorAssembler) alimente trois modèles :

- **Régression linéaire** régularisée (ElasticNet) : modèle de référence, interprétable par ses coefficients ;
- **Random Forest** : moyenne d'arbres indépendants, robuste au bruit, capte les interactions ;
- **Gradient Boosted Trees** : boosting d'arbres, l'équivalent Spark ML de XGBoost.

Les hyperparamètres de chaque modèle sont choisis par validation croisée à 3 plis (CrossValidator) sur l'apprentissage. Une référence naïve, qui prédit toujours la médiane, sert de point de comparaison.

4. Résultats

4.1 Mots et compétences les plus fréquents

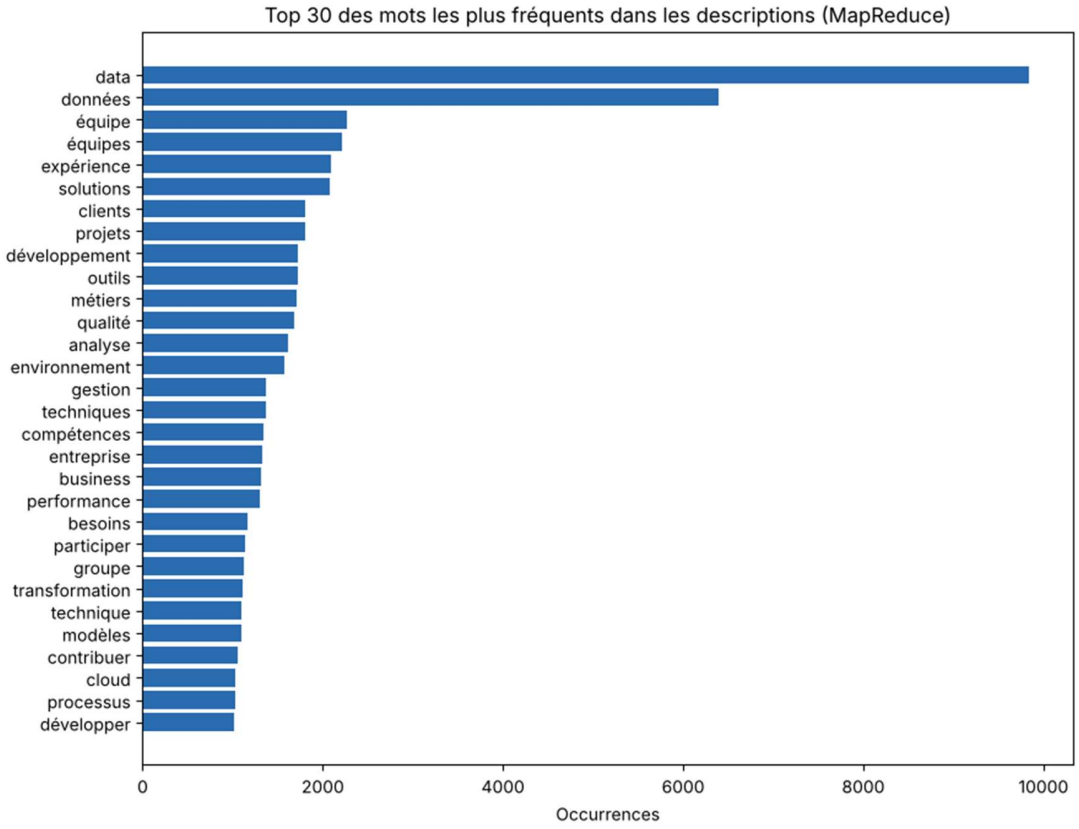


Figure 1 : les 30 mots les plus fréquents des descriptions (MapReduce)

Le vocabulaire dominant (« data », « données », « équipe(s) », « clients », « projets », « qualité ») montre que les annonces insistent autant sur le travail en équipe et la relation métier que sur la technique. Côté compétences, **SQL** (38 % des offres) et **Python** (34 %) dominent, suivis de Power BI (24 %), R (22 %) et du machine learning (20 %). Les compétences liées à l'IA générative (NLP / LLM) apparaissent déjà dans 14 % des offres.

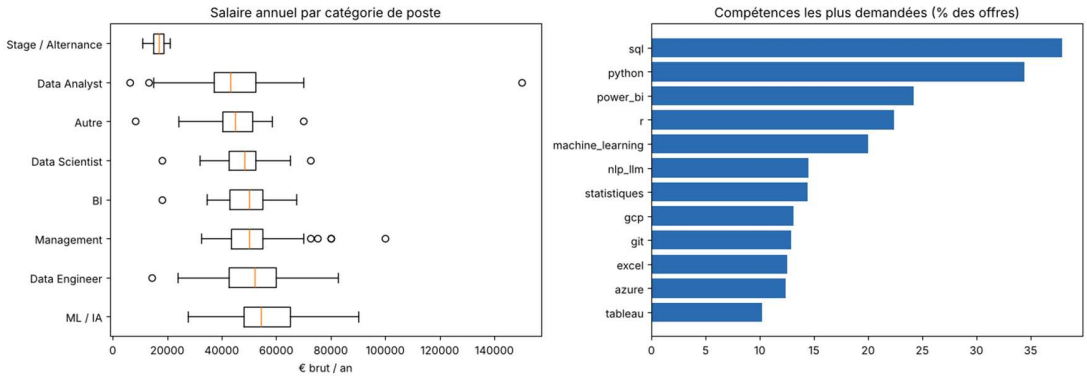


Figure 2 : salaire annuel par catégorie de poste et compétences les plus demandées

Catégorie	Offres avec salaire	Salaire médian
ML / IA	42	54 000 €
Data Engineer	94	51 500 €
Management	51	50 000 €
BI	15	50 000 €
Data Scientist	20	47 500 €
Data Analyst	86	43 000 €
Stage / Alternance	21	16 800 €

4.2 Comparaison des modèles

Modèle	RMSE CV (3 plis)	RMSE test	MAE test	R^2 test
Référence (médiane)	–	13 832 €	10 047 €	0,00
Régression linéaire	11 772 €	10 450 €	7 619 €	0,43
Random Forest	**11 588 €**	**10 002 €**	7 624 €	**0,48**
Gradient Boosted Trees	13 111 €	11 030 €	8 536 €	0,36

Les trois modèles battent nettement la référence. Le **Random Forest** est le meilleur, sur le test comme en validation croisée. Le GBT ne fait pas mieux : avec moins de 300 offres d'apprentissage, le boosting sur-apprend vite, et la validation croisée a retenu sa configuration la plus prudente (profondeur 3, pas de 0,05). La régression linéaire reste proche, signe que l'essentiel de l'information tient en quelques effets additifs.

4.3 Interprétabilité

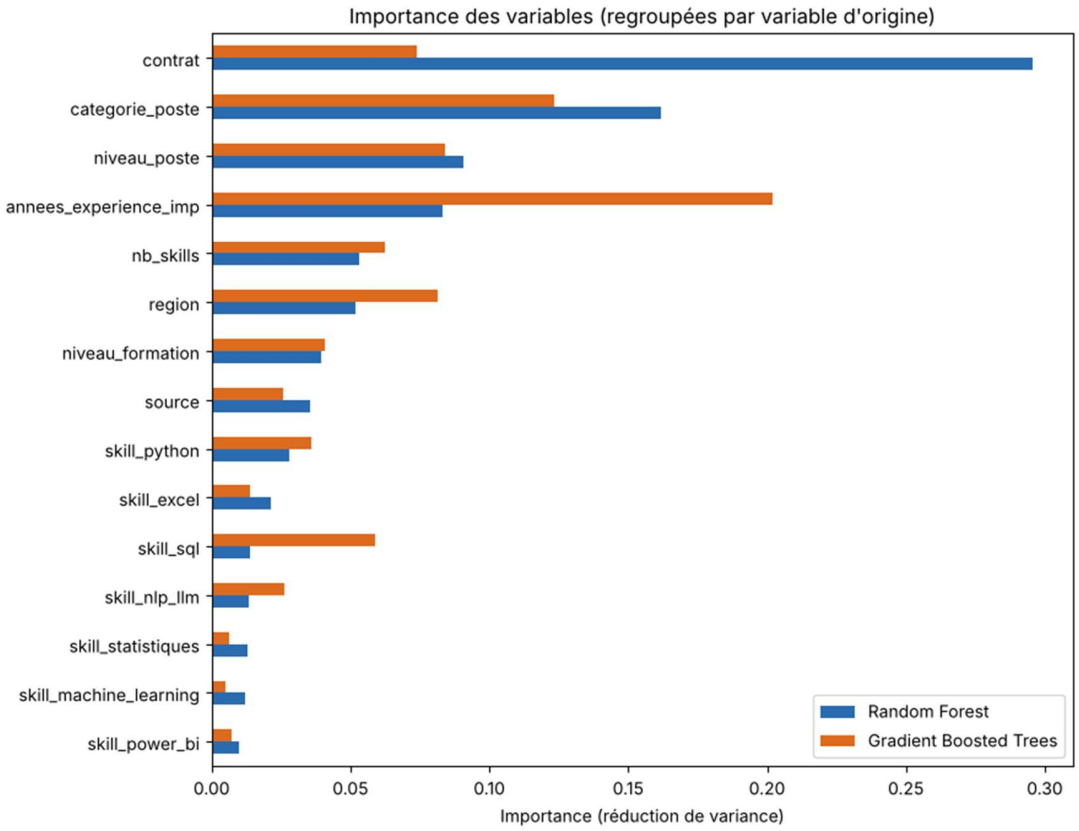


Figure 3 : importance des variables, modalités one-hot regroupées par variable d'origine

Les importances des modèles d'arbres (réduction de variance moyenne) et les coefficients de la régression linéaire convergent :

1. **Le type de contrat** est la variable la plus prédictive : un stage ou une alternance coûte environ 23 000 € par an par rapport à la référence, un CDI en apporte 10 000.
2. **La catégorie de poste** : les profils ML / IA et Data Engineer sont mieux payés que les Data Analysts.
3. **La séniorité et l'expérience** : un intitulé « Lead / Manager » ajoute environ 9 500 €, un intitulé « Junior » en retire 12 000.
4. Dans une moindre mesure, la **région** et le **niveau de diplôme** (dans la régression, un Bac+3/4 est associé à environ 8 600 € de moins par an).

Les compétences prises une à une pèsent moins : elles reflètent surtout le métier. Les outils du cloud et de la « modern data stack » (dbt, Databricks, Spark) et l'IA générative sont associés à des salaires plus élevés, Excel à des postes moins rémunérés.

5. Collaboration et versioning

Le code est versionné dans un dépôt GitHub privé (MarieJoelleWayou/BigData2026). Le travail est développé sur une branche dédiée, projet_marché_emploi, puis intégré à main par Pull Request, conformément au workflow demandé. Les identifiants de l'API sont exclus du dépôt par le .gitignore. L'historique des commits retrace les étapes du projet : notebook initial, scraper WTTJ, puis traitement Spark et modélisation.

6. Analyse critique de l'usage de l'IA

Un assistant d'IA générative (Claude) a été utilisé pendant le projet : pour déboguer le scraper, écrire une première version du code PySpark (nettoyage, MapReduce, pipelines ML) et relire la cohérence de l'ensemble.

Apports : L'assistant a accéléré l'écriture du code « mécanique » (pipelines PySpark ML, validation croisée, mise en forme des résultats) et a aidé à diagnostiquer des bugs (clés JSON mal orthographiées, compétences restées sous forme de listes, salaire extrait avec le « sur 12 mois »). Il a aussi permis d'exécuter la partie Spark dans un environnement où Java et PySpark étaient installés.

Limites constatées.

- **La vérification humaine reste indispensable.** Le problème des intitulés WTTJ, stockés en morceaux alors que le vrai titre était dans l'identifiant, a été repéré en examinant les données. Un code qui s'exécute sans erreur peut produire des données fausses ; seule l'inspection des sorties le révèle.
- **Le code proposé dépend du contexte fourni.** Les sélecteurs HTML d'un site ou les règles de conversion des salaires ne peuvent pas être devinés : il a fallu examiner les vrais libellés (« Mensuel de 26000 Euros », « EUR 1200 - 1400 / an ») pour écrire des règles justes.

- **Les choix de modélisation restent à justifier par l'analyste.** Seuils de salaire, regroupement des catégories, filtrage du périmètre : ce sont des décisions qui influencent les résultats et que l'IA ne peut pas valider seule.
- **Risque de dépendance.** Comprendre chaque étape (pourquoi reduceByKey plutôt que groupByKey, pourquoi ajuster l'imputation sur l'apprentissage seulement) est nécessaire pour défendre le travail et le maintenir.

Éthique et cadre de la collecte. L'API France Travail est un canal officiel, utilisé dans le respect de ses quotas. Le scraping de WTTJ est limité à un faible volume, avec des pauses entre les requêtes, et ne collecte aucune donnée personnelle de candidats. Les conditions d'utilisation du site devraient être vérifiées avant toute collecte régulière.

7. Limites et perspectives

- Seules 20 % des offres affichent un salaire. L'échantillon est petit et biaisé (les entreprises qui affichent un salaire ne sont pas représentatives), d'où un R^2 autour de 0,5 et des métriques qui varient selon le découpage apprentissage / test.
- Les coefficients linéaires sont instables pour les compétences rares ou corrélées entre elles (SAS, Hadoop).
- La détection des compétences par mots-clés pourrait être remplacée par une vectorisation TF-IDF ou des embeddings des descriptions, directement exploitables dans PySpark ML.
- Le scraper WTTJ doit être adapté à la nouvelle structure du site, idéalement en interrogeant l'API de recherche qu'il utilise plutôt que le HTML.
- Une collecte planifiée (chaque semaine, par exemple) permettrait de suivre l'évolution de la demande de compétences, avec un partitionnement par date de collecte devenu pertinent à ce volume.